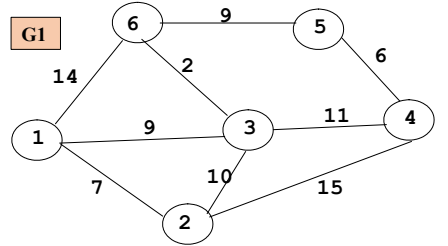


Examen – Graphes et Optimisation Combinatoire

EXERCICE 1 SUR L'ALGORITHME DE DIJKSTRA (4.5 PTS=2.0+0.25+2.25)

Considérez un graphe valué G1 représentant un réseau de transport entre plusieurs villes. Chaque sommet représente une ville et chaque arête représente une route entre deux villes, avec un coût associé à chaque arête. Vous devez appliquer l'algorithme de Dijkstra pour trouver les distances minimales des chemins allant du sommet 1 à tous les autres sommets du graphe.

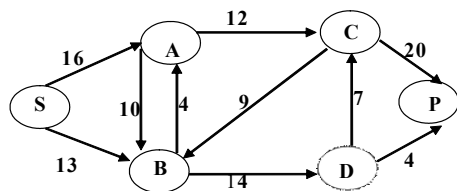


- 1) Appliquez l'algorithme de Dijkstra pour trouver les distances minimales des chemins partant du sommet 1 à tous les autres sommets du graphe.
- 2) Dessinez l'arborescence des plus courts chemins depuis le sommet de départ.
- 3) Supposons que le graphe soit représenté par une **liste d'adjacence** et que la **file de priorité** soit utilisée comme structure de données intermédiaire. Écrivez l'algorithme de Dijkstra en langage algorithmique ou en C, en utilisant ces structures.

EXERCICE 2 SUR L'ALGORITHME DE FORD & FULKERSON (5.5 PTS=2.0+3.5)

Considérez un graphe orienté G2 représentant un réseau de flux, où chaque arc a une capacité de transport maximale.

- 1) En utilisant **la stratégie du parcours en largeur** ; **appliquer étape par étape** l'algorithme de Ford & Fulkerson pour déterminer le flot maximum.



- 2) En supposant que le graphe est représenté par une **liste d'adjacence** ; et en utilisant **la stratégie du parcours en largeur** ; écrivez l'algorithme de Ford & Fulkerson en langage algorithmique ou en C en écrivant le code des algorithmes ci-dessous :

```
//Création Graphe d'écart initial à partir d'un graphe g donné
ListAdjacence creatGrapheEcart(ListAdjacence g) ;
```

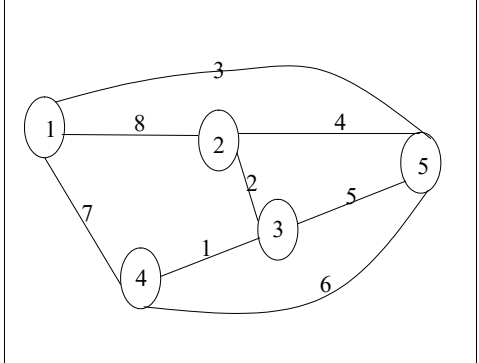
```
//Détermination de l'existence d'un chemin simple « parent » dans le
graphe //d'écart entre S et P et basée sur la méthode du parcours en
largeur
int cheminSimple(int S, int P, ListAdjacence ge, int *visite, int
*parent);
```

```
//Calcule la capacité résiduelle par rapport au chemin « parent »
entre S et P
int capaciteResiduel(int S, int P, int *parent, ListAdjacence ge) ;
```

```
//Retourne le flot Max entre S et P dans un graphe g donne
int fordFulkerson(int S, int P, ListAdjacence g);
```

EXERCICE 3 ARBRE COUVRANT DE COÛT MINIMUM (3.75 PTS=0.5+0.25+1.5+1.5)

- 1) Qu'est-ce qu'un arbre couvrant de cout minimum dans un graphe ?
- 2) Quelle condition doit être remplie pour garantir qu'il n'existe qu'un seul arbre couvrant de poids minimum dans un graphe donné ?
- 3) En plus de la phase d'initialisation, donner le résultat des **deux** premières itérations lorsqu'on applique l'algorithme de **Prim** à partir du sommet **1** au graphe ci-contre.
- 4) Utilisant une liste chaînée triée comme structure intermédiaire ; appliquer l'algorithme de **Kruskal** au graphe ci-contre.



EXERCICE 4 (1.0 PTS=0.5+0.5)

Ecrire sous forme canonique, puis sous forme standard le programme linéaire ci-contre.

$$\begin{aligned} \text{Max } F &= 3x_1 - 4x_2 + 5x_3 \\ \begin{cases} 4x_1 - 2x_2 + 3x_3 &\geq 12 \\ -5x_1 - 2x_2 + 7x_3 &\leq 3 \\ 3x_1 + x_2 + 9x_3 &= 13 \\ x_1 \in \mathbb{R}, x_2 \geq 0, x_3 \geq 0 \end{cases} \end{aligned}$$

EXERCICE 5 (2.00PTS=0.75+0.75+0.5)

Soit le programme linéaire ci-contre :

- 1) Décrire les différentes étapes de la résolution d'un programme linéaire par la méthode graphique.
- 2) Hachurer le domaine des contraintes.
- 3) Déterminer la solution optimale : valeur de F ainsi que ceux de x_1 et x_2 . (**Justifier votre réponse**).

$$\begin{aligned} \text{Max } F(x_1, x_2) &= 2x_1 + x_2 \\ \begin{cases} x_1 + x_2 &\leq 6 \\ x_1 &\leq 3 \\ x_1 + x_2 &\geq 3 \\ x_1, x_2 &\geq 0 \end{cases} \end{aligned}$$

EXERCICE 6 (3.25PTS=1.25+2.0)

- 1) Décrire les différentes étapes de la résolution d'un programme linéaire par **l'algorithme du simplexe** d'une manière générale.
- 2) Résoudre le programme ci-contre par la méthode des tableaux du simplexe.

$$\begin{aligned} \text{Max } F(x_1, x_2) &= -3x_1 + 5x_2 \\ \begin{cases} -x_1 + x_2 &\leq 3 \\ 2x_1 + x_2 &\leq 6 \\ 2x_1 - x_2 &\leq 2 \\ x_1, x_2 &\geq 0 \end{cases} \end{aligned}$$

Note : on suppose connu les structures, les primitives et les algorithmes des structures ci-dessus (voir annexe).

<pre>typedef struct cellule{ int somAdj; int cout; struct cellule *suivant; }*ListeAdj ;</pre>	<pre>typedef struct{ int nbSom; Liste *tab; }ListAdjacence;</pre>	<pre>typedef struct{ int tailleMax; int taille; int *tab; }TasMin;</pre>
--	---	--

```
int somAdj(ListeAdj l);
int estVideLAdj(ListeAdj l);
ListeAdj initLAdj();
int cout(ListeAdj l);
ListeAdj suivant(ListeAdj l) ;
```

```
int nbSom(ListAdjacence g);
ListeAdj iEmeEltsLAdj(int s, ListAdjacence g);
ListAdjacence allocSLA(int nbSom);
ListAdjacence libSLA(ListAdjacence g) ;
```

```
TasMin allocMemTas(int nbSom);
TasMin libMemTas(TasMin M);
TasMin initTas(TasMin M);
int estVideTas(TasMin M);
```

```
int minTas(TasMin M) ;
TasMin inserTas(int t, int *D, TasMin M); // Priorité est fonction
TasMin suppMinTas(int *D, TasMin M); // des distances D[t]
```

Algorithmes sur les tableaux

```
TElement *allocTab(int nMax) ;
TElement *libTab(TElement *) ;
void initTab(int n, TElement val, TElement *tab);
```

Primitives sur les Files.

```
File initF();
TElement sommetF(File f);
File enfiler(TElement elt, File f);
File defiler(File f);
```